MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión A del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_romero_paula
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/romero_paula/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
¿Qué diferencia hay entre una clase y un objeto? Usa la idea de plano e instancia.

Tu respuesta

```
la clase es un molde para el que se usa para crear un objeto
```

Corrección. Bien la idea de la clase como molde para crear objetos. Falta decir que el objeto es una instancia concreta de ese plano, con sus propios valores en los atributos.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué es la firma de un método y qué partes la forman? Explica si incluye el tipo de retorno.

Tu respuesta

```
la firma es un modo en que se llama una variable
Ej. precio (int)
```

Corrección. La respuesta confunde la firma con la forma de nombrar una variable. La firma de un método es su nombre más los tipos de sus parámetros, en orden, y no incluye el tipo de retorno.

Pregunta 0.3 · Preguntas abiertas

Pregunta

Compara la anatomía de una variable local y un atributo de instancia: dónde se declaran, valor inicial y cuánto viven.

Tu respuesta

```
Se declaran en partes distintas del codigo, y viven dento de la funcion que se este procesando con esa variable
```

> **Corrección.** Es válido que la variable local vive dentro del método que se ejecuta. Falta lo central: la local se declara en un bloque y hay que asignarle valor antes de leerla; el atributo se declara en la clase, fuera de los métodos, toma un valor por defecto y dura lo que vive el objeto.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Qué significa null en una referencia? Explica qué ocurre al llamar un método sobre esa referencia.

Tu respuesta

```
null es que no tiene valor. ocurre que el programa puede usar esa null para basarse en hacer funciones que le pidas pero sabiendo que esa variable no tiene null dentro de esta.
```

> **Corrección.** Se acerca al decir que null es que no tiene valor: más preciso, la referencia no apunta a ningún objeto. Lo que sigue es confuso y no dice lo principal: llamar un método sobre una referencia null hace que el programa truene al ejecutarse (NullPointerException).

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿Cuándo conviene usar for-each para recorrer un arreglo y cuándo necesitas un recorrido con índice?

Tu respuesta

```
for each puedes seleccionar un valor especifico sin importar su lugar y el indice es necesario cuando necesitas saber la ubicaciòn de un dato del arreglo.
```

> **Corrección.** Bien que el recorrido con índice hace falta cuando se necesita la posición del dato. Lo de for-each es impreciso: no sirve para elegir un valor específico, sino para recorrer todos los valores cuando no importa la posición (por ejemplo, para sumarlos).

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
StringBuilder ruta = new StringBuilder("S");
StringBuilder alias = ruta;
for (int i = 1; i <= 3; i++) {
```

```
    alias.append(i / 2);
    System.out.println(ruta);
}
String texto = "S" + 2 + 1;
System.out.println(texto);
double mitad = (double) (7 / 2);
System.out.println(mitad);
```

Tu respuesta

```
1 2 3
0 S 0
S12
3.5
```

Corrección. La salida correcta es S0 / S01 / S011 / S21 / 3.0: alias y ruta apuntan al mismo StringBuilder, que va agregando 1/2=0, 2/2=1 y 3/2=1. En S12 sí ves que el resultado es un texto con S, pero "S"+2+1 concatena en orden y da S21. En 3.5 sí ves que imprime un double, pero 7/2 se divide como entero antes del cast y da 3.0. Las tres primeras líneas no corresponden y falta una línea.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
double medida = 6.8;
int unidades = medida;
System.out.println(unidades);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
porque el valor 6.8 pasa de ser una variable double a ser un variable entera
```

Corrección. La clasificación es correcta (No compila) y señala bien el paso de double a int. Para la causa completa faltó decir que Java no hace sola esa conversión porque se pierde información; se necesita un cast explícito.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] canales = {2, 5, 8};
int indice = canales.length;
System.out.println(canales[indice]);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
0 1 2
2 5 8
```

Corrección. Compila pero truena: length vale 3 y los índices válidos van de 0 a 2, así que canales[3] lanza ArrayIndexOutOfBoundsException sin imprimir nada. Ni la clasificación ni la salida propuesta son correctas.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String codigo = new String("F2");
System.out.println(codigo == "F2");
System.out.println(codigo.equals("F2"));
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

```
true
true
```

Corrección. Funciona e imprime false / true: == compara si son el mismo objeto (new String crea otro) y equals compara el contenido. La clasificación es incorrecta; la segunda línea (true) sí es correcta.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada multímetro conserva su inventario y nace sin responsable. Se cuenta el total de prestados. Escribe prestar(persona) y una consulta del responsable. Supón que solo se llama prestar con un aparato disponible y un nombre válido.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿La cuenta de prestados pertenece a cada aparato o a todos? ¿Qué dato nunca cambia?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
void static(int totaloprestados);
void static.final(String prestar(persona));
for(i=0,i<totalprestados,i++){
 totalprestados = totalprestados + i;
 i++;
}
```

Justificaciones

```
El total de prestados varia de numero en cada uno, pero cuando se presta a una persona se
mantiene a quien se le presto.
```

Corrección. Bien que reconoces un contador entero de prestados con static. No hay atributos de instancia (inventario, responsable), ni constante final, constructor o métodos prestar y consulta con cuerpo coherente, y se usó un for, que estaba prohibido. Reto: el total de prestados es static (lo comparten todos los aparatos) y lo que nunca cambia es el inventario, que va como final de instancia; el responsable sí cambia.

R4 · Código con repetición

Pregunta

Un equipo guarda sus folios como enteros no negativos. Devuelve cuántos dígitos tiene folio usando while y división entre 10. El folio 0 tiene un dígito. No conviertas el número a texto.

```
static int contarDigitos(int folio)
```

Ejemplos (argumentos → retorno):

4072 → 4 · 0 → 1 · 8 → 1

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
//El programa tiene que ver el folio y de ahì dividirlo entre 10, y cuenta todas las veces que
puede dividir el folio sin que en la division se utilice un punto decimal para hacerla mas
uno.
static int contarDigitos(int folio)
while(folio!=0){
```

```
 contarDigitos = folio%10
 contarDigitos++
} if (folio=0){
  System.out.println(1);
}
return;
```

Corrección. Cabecera correcta y uso de while. El comentario habla de dividir entre 10, pero el código usa %10, no actualiza folio con folio = folio / 10 y guarda la cuenta en el nombre del método en vez de en un contador. Se ve la intención de tratar el 0, pero imprime en vez de devolver 1, usa = en lugar de == y el ciclo nunca termina. Return sin valor y con impresión. Faltan llaves y puntos y coma.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Clasifica la carga de un equipo. Usa switch clásico sobre carga / 10. Puedes validar antes con if. Devuelve el texto de la tabla. Los límites están incluidos. No imprimas ni leas del teclado.

```
static String nivelCarga(int carga)
```

Regla o valor recibido	Texto devuelto
Menor que 0 o mayor que 50	"INVALIDA"
0 a 19	"BAJA"
20 a 39	"MEDIA"
40 a 50	"ALTA"

Tu respuesta

```
static String nivelCarga(int carga)
if(0>carga>50){
  System.out.println("INVALIDA");
}else(0<=carga<=19){
  System.out.println("BAJA");
}else(20<=carga<=39){
  System.out.println("MEDIA");
}else(40<=carga<=50){
  System.out.println("ALTA");
}
```

Corrección. Cabecera correcta. Usaste if/else en vez del switch clásico que se pedía sobre carga / 10. Rangos bien agrupados e incluye 20, 40 y 50; la validación 0>carga>50 debía ser carga < 0 || carga > 50. Imprime en lugar de devolver el texto y las comparaciones encadenadas y else(cond) no son Java válido.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.